SCHOOL BUS TRACKING SYSTEM

Antigravity UI/UX Design System + Micro-Phase Prompt Book

211 micro-phase prompts • UI/UX master prompt • Antigravity workflow • ThreeUI reference strategy • verification rules

Purpose

This is the prompt companion to the detailed roadmap. Use the first sections to establish the design language and Antigravity agent behavior. Then execute one micro-phase prompt at a time.

Commercial scope

Payments, subscriptions, invoices, renewal and non-payment restrictions are postponed to the final commercial phase.

1. Antigravity Working Method

Google describes Antigravity as an agentic development environment that can operate across editor, terminal and browser, with artifacts for plans and verification. Use small, testable prompts rather than one giant build request.

☐ Open the project root in Antigravity.

☐ Keep the roadmap and this prompt book inside docs/.

☐ Create AGENTS.md using the master rules below.

☐ Use review-driven execution during early development; inspect diffs and artifacts before accepting changes.

☐ Run one micro-phase at a time.

☐ Commit after a micro-phase passes tests and visual verification.

☐ Use browser/device verification for UI work.

☐ Use parallel agents only for genuinely independent work; coordinate shared database/schema files carefully.

Recommended project files

☐ AGENTS.md — permanent agent rules.

☐ docs/product-spec.md — product requirements.

☐ docs/design-system.md — UI tokens/components.

☐ docs/feature-registry.md — Feature IDs/status.

☐ docs/architecture.md — architecture/provider abstractions.

☐ docs/adr/ — architecture decisions.

☐ docs/prompts/ — prompt library.

☐ docs/test-plan.md — acceptance/regression tests.

2. Master Agent Prompt — AGENTS.md

Build a production-grade school bus transportation platform. Do not treat this as a landing-page exercise or a static AI-generated mockup. Build a maintainable application with a clear domain model, typed contracts, reusable UI components, testable services and real state transitions.

PRODUCT
School/college bus tracking and safety platform for India. Users are school administrators, drivers and parents. Parents use WhatsApp-first workflows and a secure live web page. Drivers use a minimal Android app for GPS/background location and WhatsApp for Start Trip, End Trip, Student Search, Report Problem, Emergency and Help.

ARCHITECTURE
Backend: Node.js + TypeScript. Database: PostgreSQL + PostGIS. Driver app: Flutter Android. School dashboard: Next.js/React. Parent web: secure responsive web. Core domains: authentication, school, bus, driver, student, parent, route, pickup, trip, GPS, ETA, geofence, attendance, notification, delay, incident, SOS, reports.

ENGINEERING
Inspect before editing. Prefer small modules. Keep domain logic separate from UI. Use typed interfaces and validation. Put external providers behind adapters. Free/open-source first. Never hardcode production data or business rules into UI. Fixtures only in isolated dev/test paths. Keep secrets out of source. Add loading, empty, error, retry, unauthorized and offline states. Test business-critical logic.

UI/UX
The product should feel calm, trustworthy, precise, fast and human. Use restrained visual hierarchy, strong typography, compact data-dense layouts, clear status colors, useful motion and responsive behavior. Use ThreeUI as inspiration for interaction quality and composition, not as a clone. Avoid decorative 3D in operational screens unless it improves comprehension.

DESIGN SYSTEM
Define tokens for colors, typography, spacing, radii, shadows, borders, focus states, status states and motion. Build reusable primitives for buttons, inputs, selects, tables, cards, badges, tabs, dialogs, drawers, toasts, command menus, timelines, maps, skeletons and alerts.

SECURITY
Enforce school-level data isolation. Drivers access only assigned bus/students. Parents access only authorized transport data. Use secure sessions/tokens, HTTPS in deployment, audit logs, validation, rate limiting and safe error messages.

VERIFICATION
For every task: plan → implement → test → inspect diff → type-check/lint/build → browser/device verification where relevant → report evidence.

SCOPE
Do not add bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup. Do not build payments/subscriptions/invoices until the final commercial phase. Do not add CCTV, RFID, face recognition, payroll or full ERP unless explicitly added later.

3. UI/UX Master Design Prompt

Act as a senior product designer + design engineer. Design a cohesive operational UI for a school transportation platform, not a generic AI SaaS dashboard.

REFERENCE STRATEGY
Use https://threeui.com/browse?sort=recent as an inspiration board. Study recent patterns for component composition, typography, subtle motion, buttons, cards, sections and interaction polish. Use the ideas as references; do not copy a page, source code or asset unless licensing permits it.

VISUAL DIRECTION
- Trustworthy, calm, premium and operational.
- Neutral surfaces with one confident brand accent and restrained status colors.
- High information density without visual clutter.
- 8px-based spacing rhythm and strong typography hierarchy.
- Medium corner radius; avoid excessive pills.
- Subtle borders and shadows instead of constant glassmorphism.
- Motion explains live status, transitions and feedback.
- Maps are functional, not decorative.

SCREEN PRIORITIES
School dashboard: fleet status, delays, offline buses, incidents, today's trips.
Live map: map dominates while bus details and alerts remain accessible.
Bus/driver/student pages: searchable tables with detail drawers.
Route page: map + ordered pickup list + route state.
Attendance: fast marking workflow.
Alerts: severity, time, bus, route, location, acknowledgement and resolution.
Parent live page: simple, reassuring, mobile-first.
Driver app: large touch targets, minimal text, low cognitive load.

ANTI-AI / ANTI-HARDCODE
Do not use random gradient cards, meaningless KPI numbers, fake charts, stock illustrations, giant hero copy or decorative sections inside operational screens. Do not hardcode repeated UI. Build reusable components backed by typed data. Every interactive control must have real state. Include loading, empty and failure states. Do not leave placeholder data in production.

OUTPUT
First create a mini design brief, information architecture, token proposal, component inventory and representative screen list. Then implement the design system before individual screens. Validate desktop, tablet and mobile widths.

4. ThreeUI Reference Map

ThreeUI is best used selectively. Its current browse catalog spans UI elements, CSS/motion, Three.js, backgrounds, hero sections and templates. For this product, borrow interaction quality and composition—not flashy WebGL decoration.

Product area	Reference idea	Use
School dashboard	Data/UI composition	Dense status cards, tables, activity panels
Live map	Interactive motion	Animate live state changes; keep map primary
Alerts	Motion/status components	Severity and acknowledgement transitions
Driver app	Tactile controls	Large touch targets and one-action flows
Parent live page	Minimal composition	ETA, pickup, live state and reassurance
Forms/settings	UI/CSS components	Consistent validation, focus and confirmation
Marketing site later	Hero/sections/motion	More experimental visual language

References

ThreeUI recent browse: https://threeui.com/browse?sort=recent
ThreeUI CSS recent: https://threeui.com/css?sort=recent
ThreeUI product showcase sections: https://threeui.com/sections/product-showcases
Check licensing before copying source or assets.

5. Prompt Protocol

Each prompt below is self-contained for one micro-phase. Paste one prompt, let Antigravity plan and execute it, review the artifact/diff, verify tests and UI, commit, update the feature registry, then move forward.

☐ One micro-phase per prompt.

☐ Plan before code.

☐ Reuse existing architecture.

☐ No hardcoded production behavior.

☐ Database/API/domain logic before UI when data is involved.

☐ Loading/empty/error/offline/authorization states are part of done.

☐ Test and verify before marking Done.

☐ Document limitations and the next dependency.

MICRO-PHASE 0.1 — Master scope register

Parent phase: 0 — Project Control & Scope Lock | Category: project control

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 0: Project Control & Scope Lock
Micro-phase 0.1: Master scope register
Category: project control
Tasks: Create one master feature list; Assign unique Feature IDs; Mark each feature as Core / Safety / Supporting; Record dependencies; Create status values: Not Started / In Progress / Blocked / Done

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 0.2 — Scope exclusions

Parent phase: 0 — Project Control & Scope Lock | Category: project control

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 0: Project Control & Scope Lock
Micro-phase 0.2: Scope exclusions
Category: project control
Tasks: Keep payments, subscriptions, invoices and billing out of current build; Keep CCTV, RFID, face recognition, fuel, payroll and ERP out of current MVP; Keep custom GPS hardware out of first software build; Keep iOS driver app out of initial build

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 0.3 — Product rules

Parent phase: 0 — Project Control & Scope Lock | Category: project control

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 0: Project Control & Scope Lock
Micro-phase 0.3: Product rules
Category: project control
Tasks: WhatsApp-first for simple parent/driver actions; Parent does not need a mandatory app; Driver app remains minimal and handles GPS/background location; Free/open-source first; paid only when necessary

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 0.4 — Definition of done

Parent phase: 0 — Project Control & Scope Lock | Category: project control

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 0: Project Control & Scope Lock
Micro-phase 0.4: Definition of done
Category: project control
Tasks: Every feature must have UI/API/database behavior defined; Every critical feature must have a test case; No feature is marked Done without a real end-to-end test

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 1.1 — Developer workstation

Parent phase: 1 — Development Environment | Category: development environment

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 1: Development Environment
Micro-phase 1.1: Developer workstation
Category: development environment
Tasks: Install Git; Install VS Code; Install Node.js; Install Flutter SDK; Install Android Studio; Install Docker

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 1.2 — Repository

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 1: Development Environment
Micro-phase 1.2: Repository
Category: development environment
Tasks: Create GitHub repository; Create main branch and development branch; Create README; Add issue templates; Add environment variable template

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 1.3 — Monorepo structure

Parent phase: 1 — Development Environment | Category: development environment

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 1: Development Environment
Micro-phase 1.3: Monorepo structure
Category: development environment
Tasks: Create backend folder; Create driver-app folder; Create school-dashboard folder; Create parent-web folder; Create shared folder; Create infrastructure folder; Create docs folder; Create tests folder

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 1.4 — Local services

Parent phase: 1 — Development Environment | Category: development environment

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 1: Development Environment
Micro-phase 1.4: Local services
Category: development environment
Tasks: Run PostgreSQL locally; Enable PostGIS; Run Redis locally when needed; Create Docker Compose for local dependencies

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 2.1 — Core entities

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 2: Database & Data Model
Micro-phase 2.1: Core entities
Category: database
Tasks: Schools; Admins; Drivers; Buses; Students; Parents

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 2.2 — Transport entities

Parent phase: 2 — Database & Data Model | Category: database

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 2: Database & Data Model
Micro-phase 2.2: Transport entities
Category: database
Tasks: Routes; Pickup locations; Trips; GPS locations; Boarding events; Drop events

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 2.3 — Operations entities

Parent phase: 2 — Database & Data Model | Category: database

COPY-PASTE PROMPT

```
You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 2: Database & Data Model
Micro-phase 2.3: Operations entities
Category: database
Tasks: Notifications; Delays; Incidents; SOS events; Temporary pickup requests; Absence records; Reports; Audit logs

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.
```

MICRO-PHASE 2.4 — Location model

Parent phase: 2 — Database & Data Model | Category: database

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 2: Database & Data Model
Micro-phase 2.4: Location model
Category: database
Tasks: Store latitude/longitude; Use PostGIS geometry; Add spatial indexes; Support distance and geofence queries

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 2.5 — Database quality

Parent phase: 2 — Database & Data Model | Category: database

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 2: Database & Data Model
Micro-phase 2.5: Database quality
Category: database
Tasks: Primary/foreign keys; Unique constraints; Indexes; Created/updated timestamps; Soft-delete strategy where useful; Migration system; Seed data

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 3.1 — API skeleton

Parent phase: 3 — Backend Foundation | Category: backend

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 3: Backend Foundation
Micro-phase 3.1: API skeleton
Category: backend
Tasks: Create Node.js + TypeScript backend; Create API router; Create /health endpoint; Create configuration loader

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 3.2 — Validation

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 3: Backend Foundation
Micro-phase 3.2: Validation
Category: backend
Tasks: Request validation; Response schemas; Common error format; Input sanitization

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 3.3 — Infrastructure services

Parent phase: 3 — Backend Foundation | Category: backend

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 3: Backend Foundation
Micro-phase 3.3: Infrastructure services
Category: backend
Tasks: Database service; Logging service; Time/date utility; ID generation; File/storage abstraction

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 3.4 — API quality

Parent phase: 3 — Backend Foundation | Category: backend

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 3: Backend Foundation
Micro-phase 3.4: API quality
Category: backend
Tasks: API versioning; Rate-limit abstraction; Request IDs; Central error handler

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 4.1 — Admin authentication

Parent phase: 4 — Authentication & Authorization | Category: security/authentication

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 4: Authentication & Authorization
Micro-phase 4.1: Admin authentication
Category: security/authentication
Tasks: Login; Logout; Password hashing; Forgot password; Reset password; Change password; Session expiry

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 4.2 — Driver authentication

Parent phase: 4 — Authentication & Authorization | Category: security/authentication

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 4: Authentication & Authorization
Micro-phase 4.2: Driver authentication
Category: security/authentication
Tasks: Driver login; Session handling; Logout; Assigned-bus authorization

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 4.3 — Parent identity

Parent phase: 4 — Authentication & Authorization | Category: security/authentication

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 4: Authentication & Authorization
Micro-phase 4.3: Parent identity
Category: security/authentication
Tasks: WhatsApp identity mapping; Secure parent token; Secure tracking link; Token expiry/revocation

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 4.4 — Roles & isolation

Parent phase: 4 — Authentication & Authorization | Category: security/authentication

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 4: Authentication & Authorization
Micro-phase 4.4: Roles & isolation
Category: security/authentication
Tasks: School admin role; Driver role; Parent role; School-to-school data isolation; Driver restricted to assigned bus/students; Parent restricted to authorized student data

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 4.5 — Security audit

Parent phase: 4 — Authentication & Authorization | Category: security/authentication

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 4: Authentication & Authorization
Micro-phase 4.5: Security audit
Category: security/authentication
Tasks: Audit important actions; Protect secrets; HTTPS in deployed environments; Backup policy; Basic abuse/rate-limit protections

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 5.1 — School profile

Parent phase: 5 — School Onboarding & Admin | Category: school admin UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 5: School Onboarding & Admin
Micro-phase 5.1: School profile
Category: school admin UI
Tasks: School name; Address; City; State; Contact; Email; Logo

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 5.2 — Admin profile

Parent phase: 5 — School Onboarding & Admin | Category: school admin UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 5: School Onboarding & Admin
Micro-phase 5.2: Admin profile
Category: school admin UI
Tasks: Admin mobile; Login credentials; Profile; Password change; Logout

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 5.3 — Dashboard shell

Parent phase: 5 — School Onboarding & Admin | Category: school admin UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 5: School Onboarding & Admin
Micro-phase 5.3: Dashboard shell
Category: school admin UI
Tasks: Sidebar; Header; Navigation; Protected pages; Responsive layout; Empty states

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 5.4 — Admin home

Parent phase: 5 — School Onboarding & Admin | Category: school admin UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 5: School Onboarding & Admin
Micro-phase 5.4: Admin home
Category: school admin UI
Tasks: Dashboard cards; Recent activity area; Alert summary; Transport status summary

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 6.1 — Create bus

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 6: Bus Management
Micro-phase 6.1: Create bus
Category: bus management UI
Tasks: Bus number; Registration number; Bus details; Save validation

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 6.2 — Assignments

Parent phase: 6 — Bus Management | Category: bus management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 6: Bus Management
Micro-phase 6.2: Assignments
Category: bus management UI
Tasks: Assign driver; Assign route; Show assignments

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 6.3 — Bus details

Parent phase: 6 — Bus Management | Category: bus management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 6: Bus Management
Micro-phase 6.3: Bus details
Category: bus management UI
Tasks: View bus; Edit bus; Remove bus; Current location; Current trip

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 6.4 — Bus history

Parent phase: 6 — Bus Management | Category: bus management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 6: Bus Management
Micro-phase 6.4: Bus history
Category: bus management UI
Tasks: Trip history; Incident history; Attendance history

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 7.1 — Create driver

Parent phase: 7 — Driver Management | Category: driver management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 7: Driver Management
Micro-phase 7.1: Create driver
Category: driver management UI
Tasks: Name; Mobile; Photo; Login

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 7.2 — Assignment

Parent phase: 7 — Driver Management | Category: driver management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 7: Driver Management
Micro-phase 7.2: Assignment
Category: driver management UI
Tasks: Assign bus; Show assigned bus; Show assigned route

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 7.3 — Driver history

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 7: Driver Management
Micro-phase 7.3: Driver history
Category: driver management UI
Tasks: Trip history; Boarding activity; Incident history

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 7.4 — Driver profile

Parent phase: 7 — Driver Management | Category: driver management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 7: Driver Management
Micro-phase 7.4: Driver profile
Category: driver management UI
Tasks: View; Edit; Credential/session controls

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 8.1 — Create student

Parent phase: 8 — Student Management | Category: student management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 8: Student Management
Micro-phase 8.1: Create student
Category: student management UI
Tasks: Name; Class; Section; Student ID

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 8.2 — Parent/guardian data

Parent phase: 8 — Student Management | Category: student management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 8: Student Management
Micro-phase 8.2: Parent/guardian data
Category: student management UI
Tasks: Guardian name; Guardian phone; Emergency contact

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 8.3 — Transport assignment

Parent phase: 8 — Student Management | Category: student management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 8: Student Management
Micro-phase 8.3: Transport assignment
Category: student management UI
Tasks: Assigned bus; Pickup location; Transport status

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 8.4 — Student records

Parent phase: 8 — Student Management | Category: student management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 8: Student Management
Micro-phase 8.4: Student records
Category: student management UI
Tasks: View; Edit; Remove; Transport history

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 9.1 — Parent record

Parent phase: 9 — Parent Management & Onboarding | Category: parent management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 9: Parent Management & Onboarding
Micro-phase 9.1: Parent record
Category: parent management UI
Tasks: Parent name; Mobile; Relationship; WhatsApp status

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 9.2 — Student assignment

Parent phase: 9 — Parent Management & Onboarding | Category: parent management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 9: Parent Management & Onboarding
Micro-phase 9.2: Student assignment
Category: parent management UI
Tasks: Link authorized student; Show assignment; Communication history

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 9.3 — WhatsApp onboarding

Parent phase: 9 — Parent Management & Onboarding | Category: parent management UI

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 9: Parent Management & Onboarding
Micro-phase 9.3: WhatsApp onboarding
Category: parent management UI
Tasks: Invitation flow; Identity verification; Activation confirmation; Invalid/expired token handling

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 9.4 — Notification status

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 9: Parent Management & Onboarding
Micro-phase 9.4: Notification status
Category: parent management UI
Tasks: Track notification delivery state; Show communication history

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 10.1 — Flutter app shell

Parent phase: 10 — Driver App Foundation | Category: driver mobile app

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 10: Driver App Foundation
Micro-phase 10.1: Flutter app shell
Category: driver mobile app
Tasks: Login screen; Home screen; Today's bus; Today's route

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 10.2 — Device permissions

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 10: Driver App Foundation
Micro-phase 10.2: Device permissions
Category: driver mobile app
Tasks: Location permission; Background location permission; Notification permission where required

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 10.3 — Driver status

Parent phase: 10 — Driver App Foundation | Category: driver mobile app

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 10: Driver App Foundation
Micro-phase 10.3: Driver status
Category: driver mobile app
Tasks: GPS status; Network status; Battery status; Trip status

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 10.4 — Reliability

Parent phase: 10 — Driver App Foundation | Category: driver mobile app

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 10: Driver App Foundation
Micro-phase 10.4: Reliability
Category: driver mobile app
Tasks: Reconnect logic; App lifecycle handling; Last-known state; Local pending queue

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 11.1 — Location capture

Parent phase: 11 — GPS Capture & Ingestion | Category: GPS/location

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 11: GPS Capture & Ingestion
Micro-phase 11.1: Location capture
Category: GPS/location
Tasks: Latitude; Longitude; Timestamp; Speed; Accuracy; Battery; Network

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 11.2 — Upload

Parent phase: 11 — GPS Capture & Ingestion | Category: GPS/location

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 11: GPS Capture & Ingestion
Micro-phase 11.2: Upload
Category: GPS/location
Tasks: Send location to backend; Validate payload; Retry failed upload; Avoid duplicate points

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 11.3 — GPS health

Parent phase: 11 — GPS Capture & Ingestion | Category: GPS/location

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 11: GPS Capture & Ingestion
Micro-phase 11.3: GPS health
Category: GPS/location
Tasks: Detect stale GPS; Detect poor accuracy; Detect internet loss; Store last known location

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 11.4 — Backend processing

Parent phase: 11 — GPS Capture & Ingestion | Category: GPS/location

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 11: GPS Capture & Ingestion
Micro-phase 11.4: Backend processing
Category: GPS/location
Tasks: Validate source; Store raw location; Update current bus position; Prepare real-time feed

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 12.1 — Current bus state

Parent phase: 12 — Real-Time Live Tracking | Category: real-time tracking

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 12: Real-Time Live Tracking
Micro-phase 12.1: Current bus state
Category: real-time tracking
Tasks: Current location; Last update time; Speed; Trip state

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 12.2 — Real-time transport

Parent phase: 12 — Real-Time Live Tracking | Category: real-time tracking

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 12: Real-Time Live Tracking
Micro-phase 12.2: Real-time transport
Category: real-time tracking
Tasks: WebSocket/SSE or equivalent; Reconnect; Subscription by school/bus

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 12.3 — School map feed

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 12: Real-Time Live Tracking
Micro-phase 12.3: School map feed
Category: real-time tracking
Tasks: Bus marker; Driver; Route; Trip status; Last update

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 12.4 — Parent live feed

Parent phase: 12 — Real-Time Live Tracking | Category: real-time tracking

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 12: Real-Time Live Tracking
Micro-phase 12.4: Parent live feed
Category: real-time tracking
Tasks: Authorized bus only; Secure live location access; Graceful offline state

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 13.1 — Map abstraction

Parent phase: 13 — Map & Routing Layer | Category: maps/routing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 13: Map & Routing Layer
Micro-phase 13.1: Map abstraction
Category: maps/routing
Tasks: Create MapService interface; Create RoutingService interface; Keep provider-specific code isolated

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 13.2 — Free-first maps

Parent phase: 13 — Map & Routing Layer | Category: maps/routing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 13: Map & Routing Layer
Micro-phase 13.2: Free-first maps
Category: maps/routing
Tasks: OpenStreetMap data; Map rendering option; Geocoding option; Routing option

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 13.3 — Routing

Parent phase: 13 — Map & Routing Layer | Category: maps/routing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 13: Map & Routing Layer
Micro-phase 13.3: Routing
Category: maps/routing
Tasks: OSRM/Valhalla evaluation; Route distance; Road path; Fallback behavior

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 13.4 — Provider fallback

Parent phase: 13 — Map & Routing Layer | Category: maps/routing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 13: Map & Routing Layer
Micro-phase 13.4: Provider fallback
Category: maps/routing
Tasks: Document when paid maps may be used; Keep provider switch possible; Track map API usage

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 14.1 — Parent location sharing

Parent phase: 14 — Dynamic Pickup Engine | Category: dynamic pickup

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 14: Dynamic Pickup Engine
Micro-phase 14.1: Parent location sharing
Category: dynamic pickup
Tasks: WhatsApp asks for location; Receive coordinates; Validate coordinates

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 14.2 — Pickup point creation

Parent phase: 14 — Dynamic Pickup Engine | Category: dynamic pickup

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 14: Dynamic Pickup Engine
Micro-phase 14.2: Pickup point creation
Category: dynamic pickup
Tasks: Find nearest suitable road; Reverse geocode; Create readable address; Store coordinates

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 14.3 — Student mapping

Parent phase: 14 — Dynamic Pickup Engine | Category: dynamic pickup

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 14: Dynamic Pickup Engine
Micro-phase 14.3: Student mapping
Category: dynamic pickup
Tasks: Assign pickup to student; Show pickup on map; Update pickup

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 14.4 — Safety/quality checks

Parent phase: 14 — Dynamic Pickup Engine | Category: dynamic pickup

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 14: Dynamic Pickup Engine
Micro-phase 14.4: Safety/quality checks
Category: dynamic pickup
Tasks: Reject clearly invalid locations; Detect duplicate/nearby pickup points; Allow school to edit/change pickup

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 15.1 — Create route

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 15: Route Management
Micro-phase 15.1: Create route
Category: route management
Tasks: Route name; Assign bus; Assign driver

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 15.2 — Pickup sequence

Parent phase: 15 — Route Management | Category: route management

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 15: Route Management
Micro-phase 15.2: Pickup sequence
Category: route management
Tasks: Add pickup locations; Order pickup sequence; Edit sequence; Remove pickup

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 15.3 — Route map

Parent phase: 15 — Route Management | Category: route management

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 15: Route Management
Micro-phase 15.3: Route map
Category: route management
Tasks: Draw route; Show pickup points; Show route details

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 15.4 — Route history

Parent phase: 15 — Route Management | Category: route management

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 15: Route Management
Micro-phase 15.4: Route history
Category: route management
Tasks: Track changes; Show previous route state where required

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 16.1 — Trip lifecycle

Parent phase: 16 — Trip Engine | Category: trip engine

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 16: Trip Engine
Micro-phase 16.1: Trip lifecycle
Category: trip engine
Tasks: NOT_STARTED; STARTED; IN_PROGRESS; ARRIVED_SCHOOL; COMPLETED

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 16.2 — Trip creation

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 16: Trip Engine
Micro-phase 16.2: Trip creation
Category: trip engine
Tasks: Create today's trip; Attach bus; Attach driver; Attach route

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 16.3 — Trip start

Parent phase: 16 — Trip Engine | Category: trip engine

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 16: Trip Engine
Micro-phase 16.3: Trip start
Category: trip engine
Tasks: Start timestamp; Activate tracking session; Update dashboard

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 16.4 — Trip end

Parent phase: 16 — Trip Engine | Category: trip engine

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 16: Trip Engine
Micro-phase 16.4: Trip end
Category: trip engine
Tasks: End timestamp; Stop trip state; Lock trip attendance; Generate summary

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 16.5 — Idempotency

Parent phase: 16 — Trip Engine | Category: trip engine

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 16: Trip Engine
Micro-phase 16.5: Idempotency
Category: trip engine
Tasks: Prevent duplicate starts; Prevent duplicate ends; Handle retries safely

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 17.1 — Menu

Parent phase: 17 — Driver WhatsApp Core | Category: driver WhatsApp

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 17: Driver WhatsApp Core
Micro-phase 17.1: Menu
Category: driver WhatsApp
Tasks: Start Trip; End Trip; Student Search; Report Problem; Emergency; Help

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 17.2 — Start Trip flow

Parent phase: 17 — Driver WhatsApp Core | Category: driver WhatsApp

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 17: Driver WhatsApp Core
Micro-phase 17.2: Start Trip flow
Category: driver WhatsApp
Tasks: Identify driver; Identify assigned bus; Create/start trip; Activate GPS session; Trigger parent notifications; Update school dashboard

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 17.3 — End Trip flow

Parent phase: 17 — Driver WhatsApp Core | Category: driver WhatsApp

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 17: Driver WhatsApp Core
Micro-phase 17.3: End Trip flow
Category: driver WhatsApp
Tasks: Identify active trip; End trip; Stop session; Lock attendance; Generate trip summary

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 17.4 — Error handling

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 17: Driver WhatsApp Core
Micro-phase 17.4: Error handling
Category: driver WhatsApp
Tasks: No assigned bus; No active trip; Duplicate command; Unknown command; Temporary WhatsApp/API failure

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 18.1 — Student list

Parent phase: 18 — Driver Attendance | Category: attendance

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 18: Driver Attendance
Micro-phase 18.1: Student list
Category: attendance
Tasks: Show registered students for assigned bus; Show pickup details

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 18.2 — Boarding

Parent phase: 18 — Driver Attendance | Category: attendance

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 18: Driver Attendance
Micro-phase 18.2: Boarding
Category: attendance
Tasks: Boarded; Not boarded; Timestamp; Location

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 18.3 — Drop

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 18: Driver Attendance
Micro-phase 18.3: Drop
Category: attendance
Tasks: Dropped; Timestamp; Location

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 18.4 — History

Parent phase: 18 — Driver Attendance | Category: attendance

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 18: Driver Attendance
Micro-phase 18.4: History
Category: attendance
Tasks: Trip/date history; Student journey history; Correction rules

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 19.1 — Search

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 19: Driver Student Search
Micro-phase 19.1: Search
Category: student search
Tasks: Search by student name; Restrict results to assigned bus

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 19.2 — Student result

Parent phase: 19 — Driver Student Search | Category: student search

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 19: Driver Student Search
Micro-phase 19.2: Student result
Category: student search
Tasks: Student name; Parent name; Parent phone; Pickup address; Pickup location; Bus; Route

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 19.3 — Actions

Parent phase: 19 — Driver Student Search | Category: student search

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 19: Driver Student Search
Micro-phase 19.3: Actions
Category: student search
Tasks: Call parent; Open map/navigation; Return to menu

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 20.1 — ETA inputs

Parent phase: 20 — ETA Engine | Category: ETA

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 20: ETA Engine
Micro-phase 20.1: ETA inputs
Category: ETA
Tasks: Current bus position; Route; Pickup locations; Road distance; Current/estimated speed

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 20.2 — ETA calculation

Parent phase: 20 — ETA Engine | Category: ETA

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 20: ETA Engine
Micro-phase 20.2: ETA calculation
Category: ETA
Tasks: Bus-to-pickup ETA; Next pickup ETA; Remaining route ETA

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 20.3 — ETA refresh

Parent phase: 20 — ETA Engine | Category: ETA

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 20: ETA Engine
Micro-phase 20.3: ETA refresh
Category: ETA
Tasks: Refresh after GPS updates; Smooth noisy estimates; Handle stopped bus

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 20.4 — Parent ETA

Parent phase: 20 — ETA Engine | Category: ETA

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 20: ETA Engine
Micro-phase 20.4: Parent ETA
Category: ETA
Tasks: Generate parent-specific ETA; Show last update; Handle unavailable ETA

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 21.1 — Pickup geofence

Parent phase: 21 — Geofencing | Category: geofencing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 21: Geofencing
Micro-phase 21.1: Pickup geofence
Category: geofencing
Tasks: Create radius/shape; Detect entry; Detect exit

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 21.2 — School geofence

Parent phase: 21 — Geofencing | Category: geofencing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 21: Geofencing
Micro-phase 21.2: School geofence
Category: geofencing
Tasks: Create school geofence; Detect school arrival

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 21.3 — Event handling

Parent phase: 21 — Geofencing | Category: geofencing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 21: Geofencing
Micro-phase 21.3: Event handling
Category: geofencing
Tasks: Debounce repeated events; Store timestamp/location; Trigger workflows

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 22.1 — Parent menu

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 22: Parent WhatsApp
Micro-phase 22.1: Parent menu
Category: parent WhatsApp
Tasks: Track My Bus; My ETA; My Child; My Pickup; Notifications; Help

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 22.2 — Track My Bus

Parent phase: 22 — Parent WhatsApp | Category: parent WhatsApp

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 22: Parent WhatsApp
Micro-phase 22.2: Track My Bus
Category: parent WhatsApp
Tasks: Current bus; Location; Route; Pickup; ETA; Trip status; Secure live link

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 22.3 — My ETA

Parent phase: 22 — Parent WhatsApp | Category: parent WhatsApp

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 22: Parent WhatsApp
Micro-phase 22.3: My ETA
Category: parent WhatsApp
Tasks: Return latest ETA; Return last update; Handle unavailable ETA

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 22.4 — My Child

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 22: Parent WhatsApp
Micro-phase 22.4: My Child
Category: parent WhatsApp
Tasks: Current journey status; Boarding status; Drop status

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 22.5 — My Pickup

Parent phase: 22 — Parent WhatsApp | Category: parent WhatsApp

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 22: Parent WhatsApp
Micro-phase 22.5: My Pickup
Category: parent WhatsApp
Tasks: View pickup; Request pickup update flow

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 22.6 — Help

Parent phase: 22 — Parent WhatsApp | Category: parent WhatsApp

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 22: Parent WhatsApp
Micro-phase 22.6: Help
Category: parent WhatsApp
Tasks: FAQ; Support contact; Issue reporting

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 23.1 — Secure entry

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 23: Parent Live Web Page
Micro-phase 23.1: Secure entry
Category: parent live web
Tasks: Token validation; Expiry; Revocation

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 23.2 — Live view

Parent phase: 23 — Parent Live Web Page | Category: parent live web

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 23: Parent Live Web Page
Micro-phase 23.2: Live view
Category: parent live web
Tasks: Bus marker; Route; Pickup; ETA; Bus number; Trip status; Last update

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 23.3 — Failure states

Parent phase: 23 — Parent Live Web Page | Category: parent live web

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 23: Parent Live Web Page
Micro-phase 23.3: Failure states
Category: parent live web
Tasks: Bus offline; Trip ended; Invalid link; Stale location

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 24.1 — Event model

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 24: Notification Engine
Micro-phase 24.1: Event model
Category: notifications
Tasks: Define notification events; Define recipients; Define priority

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 24.2 — Parent events

Parent phase: 24 — Notification Engine | Category: notifications

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 24: Notification Engine
Micro-phase 24.2: Parent events
Category: notifications
Tasks: Bus started; Approaching; 10-minute alert; 3-minute/nearby alert; Reached pickup; Child boarded; Child dropped; Trip completed

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 24.3 — Operational events

Parent phase: 24 — Notification Engine | Category: notifications

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 24: Notification Engine
Micro-phase 24.3: Operational events
Category: notifications
Tasks: Delay; GPS unavailable; Route deviation; Emergency; Unusual stop

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 24.4 — Delivery

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 24: Notification Engine
Micro-phase 24.4: Delivery
Category: notifications
Tasks: WhatsApp; Push where appropriate; Dashboard alerts; Retry/failure state

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 24.5 — Anti-spam

Parent phase: 24 — Notification Engine | Category: notifications

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 24: Notification Engine
Micro-phase 24.5: Anti-spam
Category: notifications
Tasks: Deduplicate events; Throttle repeated alerts; Only notify affected parents

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 25.1 — Expected timeline

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 25: Smart Delay Engine
Micro-phase 25.1: Expected timeline
Category: delay intelligence
Tasks: Calculate expected ETA for each pickup; Store expected arrival

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 25.2 — Actual timeline

Parent phase: 25 — Smart Delay Engine | Category: delay intelligence

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 25: Smart Delay Engine
Micro-phase 25.2: Actual timeline
Category: delay intelligence
Tasks: Compare current ETA; Calculate delay minutes

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 25.3 — Affected parent logic

Parent phase: 25 — Smart Delay Engine | Category: delay intelligence

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 25: Smart Delay Engine
Micro-phase 25.3: Affected parent logic
Category: delay intelligence
Tasks: Identify upcoming affected pickups; Exclude already-served pickups; Identify parents still waiting

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 25.4 — Delay notifications

Parent phase: 25 — Smart Delay Engine | Category: delay intelligence

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 25: Smart Delay Engine
Micro-phase 25.4: Delay notifications
Category: delay intelligence
Tasks: Send delay alert; Update ETA; Send recovery update when appropriate

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 25.5 — Spam control

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 25: Smart Delay Engine
Micro-phase 25.5: Spam control
Category: delay intelligence
Tasks: Avoid repeated identical delay messages; Notify only on meaningful change

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 26.1 — Detection

Parent phase: 26 — Long Stop Detection | Category: long-stop detection

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 26: Long Stop Detection
Micro-phase 26.1: Detection
Category: long-stop detection
Tasks: Detect stationary bus; Configurable threshold; Ignore GPS jitter

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 26.2 — Driver reason

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 26: Long Stop Detection
Micro-phase 26.2: Driver reason
Category: long-stop detection
Tasks: Traffic; Vehicle problem; Student issue; Road block; Other; No problem

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 26.3 — School workflow

Parent phase: 26 — Long Stop Detection | Category: long-stop detection

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 26: Long Stop Detection
Micro-phase 26.3: School workflow
Category: long-stop detection
Tasks: Create event; Notify school; Store reason; Resolve event

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 27.1 — Detection

Parent phase: 27 — GPS Offline & Recovery | Category: GPS recovery

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 27: GPS Offline & Recovery
Micro-phase 27.1: Detection
Category: GPS recovery
Tasks: Stale location threshold; Internet loss; App disconnect

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 27.2 — Response

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 27: GPS Offline & Recovery
Micro-phase 27.2: Response
Category: GPS recovery
Tasks: Store last known location; Notify school; Notify driver

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 27.3 — Recovery

Parent phase: 27 — GPS Offline & Recovery | Category: GPS recovery

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 27: GPS Offline & Recovery
Micro-phase 27.3: Recovery
Category: GPS recovery
Tasks: Reconnect automatically; Upload queued points where valid; Mark recovered; Avoid duplicate points

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 28.1 — Expected route

Parent phase: 28 — Route Deviation | Category: route deviation

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 28: Route Deviation
Micro-phase 28.1: Expected route
Category: route deviation
Tasks: Load route geometry; Determine bus position relative to route

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 28.2 — Detection

Parent phase: 28 — Route Deviation | Category: route deviation

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 28: Route Deviation
Micro-phase 28.2: Detection
Category: route deviation
Tasks: Distance threshold; Debounce; Create deviation event

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 28.3 — Response

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 28: Route Deviation
Micro-phase 28.3: Response
Category: route deviation
Tasks: School alert; Incident record; Current location; Timestamp

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 29.1 — Rule

Parent phase: 29 — Overspeed | Category: overspeed

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 29: Overspeed
Micro-phase 29.1: Rule
Category: overspeed
Tasks: Configurable speed threshold; Optional duration threshold

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 29.2 — Detection

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 29: Overspeed
Micro-phase 29.2: Detection
Category: overspeed
Tasks: Use validated GPS speed; Avoid single-point false positives

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 29.3 — Event

Parent phase: 29 — Overspeed | Category: overspeed

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 29: Overspeed
Micro-phase 29.3: Event
Category: overspeed
Tasks: Speed; Time; Location; Duration

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 29.4 — Reporting

Parent phase: 29 — Overspeed | Category: overspeed

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 29: Overspeed
Micro-phase 29.4: Reporting
Category: overspeed
Tasks: School notification; Daily report inclusion; History

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 30.1 — Trigger

Parent phase: 30 — Emergency / SOS | Category: emergency/SOS

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 30: Emergency / SOS
Micro-phase 30.1: Trigger
Category: emergency/SOS
Tasks: Driver WhatsApp emergency; Optional app emergency control

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 30.2 — Capture

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 30: Emergency / SOS
Micro-phase 30.2: Capture
Category: emergency/SOS
Tasks: Bus; Driver; GPS; Timestamp; Active trip

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 30.3 — School alert

Parent phase: 30 — Emergency / SOS | Category: emergency/SOS

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 30: Emergency / SOS
Micro-phase 30.3: School alert
Category: emergency/SOS
Tasks: Immediate alert; Live bus location; Driver details; Incident record

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 30.4 — Resolution

Parent phase: 30 — Emergency / SOS | Category: emergency/SOS

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 30: Emergency / SOS
Micro-phase 30.4: Resolution
Category: emergency/SOS
Tasks: Acknowledge; Resolve; Store incident history

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 30.5 — Scope note

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 30: Emergency / SOS
Micro-phase 30.5: Scope note
Category: emergency/SOS
Tasks: Keep software SOS separate from physical AIS-140 panic hardware

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 31.1 — Upcoming student

Parent phase: 31 — Pickup Arrival Workflow | Category: pickup arrival

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 31: Pickup Arrival Workflow
Micro-phase 31.1: Upcoming student
Category: pickup arrival
Tasks: Determine next pickup; Identify associated student

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 31.2 — Approach

Parent phase: 31 — Pickup Arrival Workflow | Category: pickup arrival

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 31: Pickup Arrival Workflow
Micro-phase 31.2: Approach
Category: pickup arrival
Tasks: Approaching notification; Driver upcoming-student view

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 31.3 — Arrival

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 31: Pickup Arrival Workflow
Micro-phase 31.3: Arrival
Category: pickup arrival
Tasks: Geofence event; Parent arrival notification; Boarding available

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 32.1 — Journey timeline

Parent phase: 32 — Student Journey Record | Category: student journey

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 32: Student Journey Record
Micro-phase 32.1: Journey timeline
Category: student journey
Tasks: Pickup; Bus arrival; Boarded/not boarded; On route; School arrival; Dropped

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 32.2 — History

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 32: Student Journey Record
Micro-phase 32.2: History
Category: student journey
Tasks: Student transport history; Trip history; Attendance history

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 32.3 — Data consistency

Parent phase: 32 — Student Journey Record | Category: student journey

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 32: Student Journey Record
Micro-phase 32.3: Data consistency
Category: student journey
Tasks: One journey per trip/student; Prevent duplicate attendance events

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 33.1 — Parent flow

Parent phase: 33 — Student Not Going Today | Category: absence

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 33: Student Not Going Today
Micro-phase 33.1: Parent flow
Category: absence
Tasks: Select student; Select date; Confirm absence

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 33.2 — System response

Parent phase: 33 — Student Not Going Today | Category: absence

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 33: Student Not Going Today
Micro-phase 33.2: System response
Category: absence
Tasks: Mark transport absence; Notify school/driver; Update driver's student list

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 33.3 — Trip behavior

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 33: Student Not Going Today
Micro-phase 33.3: Trip behavior
Category: absence
Tasks: Do not expect boarding alert for absent student; Keep audit record

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 34.1 — Request

Parent phase: 34 — Temporary Pickup Change | Category: temporary pickup

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 34: Temporary Pickup Change
Micro-phase 34.1: Request
Category: temporary pickup
Tasks: Parent chooses date; Parent shares new location; Create request

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 34.2 — School review

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 34: Temporary Pickup Change
Micro-phase 34.2: School review
Category: temporary pickup
Tasks: View request; Accept; Reject; Record decision

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 34.3 — Driver update

Parent phase: 34 — Temporary Pickup Change | Category: temporary pickup

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 34: Temporary Pickup Change
Micro-phase 34.3: Driver update
Category: temporary pickup
Tasks: Update route/pickup information; Show temporary pickup clearly

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 34.4 — Parent confirmation

Parent phase: 34 — Temporary Pickup Change | Category: temporary pickup

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 34: Temporary Pickup Change
Micro-phase 34.4: Parent confirmation
Category: temporary pickup
Tasks: Notify parent of final decision; Show effective date

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 35.1 — Summary

Parent phase: 35 — School Live Dashboard | Category: school dashboard

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 35: School Live Dashboard
Micro-phase 35.1: Summary
Category: school dashboard
Tasks: Running buses; Completed trips; Delayed; Offline; Incidents; Boarding/drop overview

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 35.2 — Bus list

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 35: School Live Dashboard
Micro-phase 35.2: Bus list
Category: school dashboard
Tasks: Bus status; Driver; Route; Trip state; Last GPS update

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 35.3 — Quick actions

Parent phase: 35 — School Live Dashboard | Category: school dashboard

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 35: School Live Dashboard
Micro-phase 35.3: Quick actions
Category: school dashboard
Tasks: Open live map; Open bus details; Open alerts

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 36.1 — Fleet map

Parent phase: 36 — School Live Map | Category: live map

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 36: School Live Map
Micro-phase 36.1: Fleet map
Category: live map
Tasks: All authorized buses; Markers; Current location

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 36.2 — Bus details

Parent phase: 36 — School Live Map | Category: live map

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 36: School Live Map
Micro-phase 36.2: Bus details
Category: live map
Tasks: Driver; Trip; Student count/attendance overview; ETA

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 36.3 — Map filters

Parent phase: 36 — School Live Map | Category: live map

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 36: School Live Map
Micro-phase 36.3: Map filters
Category: live map
Tasks: Running; Delayed; Offline; Incidents

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 37.1 — Boarding view

Parent phase: 37 — Attendance Dashboard | Category: attendance dashboard

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 37: Attendance Dashboard
Micro-phase 37.1: Boarding view
Category: attendance dashboard
Tasks: Boarded; Not boarded; Timestamp; Location

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 37.2 — Drop view

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 37: Attendance Dashboard
Micro-phase 37.2: Drop view
Category: attendance dashboard
Tasks: Dropped; Timestamp; Location

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 37.3 — Filters

Parent phase: 37 — Attendance Dashboard | Category: attendance dashboard

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 37: Attendance Dashboard
Micro-phase 37.3: Filters
Category: attendance dashboard
Tasks: Date; Bus; Route; Driver; Student

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 37.4 — History

Parent phase: 37 — Attendance Dashboard | Category: attendance dashboard

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 37: Attendance Dashboard
Micro-phase 37.4: History
Category: attendance dashboard
Tasks: Trip history; Student history

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 38.1 — Report inputs

Parent phase: 38 — Daily Transport Reports | Category: reports

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 38: Daily Transport Reports
Micro-phase 38.1: Report inputs
Category: reports
Tasks: School; Date; Bus; Driver; Route

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 38.2 — Trip metrics

Parent phase: 38 — Daily Transport Reports | Category: reports

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 38: Daily Transport Reports
Micro-phase 38.2: Trip metrics
Category: reports
Tasks: Trip start; School arrival; Expected; Boarded; Not boarded; Dropped

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 38.3 — Safety/quality metrics

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 38: Daily Transport Reports
Micro-phase 38.3: Safety/quality metrics
Category: reports
Tasks: Delay; Route deviation; Overspeed; GPS downtime; Incidents; Emergency

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 38.4 — Report generation

Parent phase: 38 — Daily Transport Reports | Category: reports

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 38: Daily Transport Reports
Micro-phase 38.4: Report generation
Category: reports
Tasks: Dashboard report; CSV export

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 39.1 — Daily summary

Parent phase: 39 — Report & Summary Delivery | Category: report delivery

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 39: Report & Summary Delivery
Micro-phase 39.1: Daily summary
Category: report delivery
Tasks: Generate daily summary; Send to authorized school recipients

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 39.2 — Weekly summary

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 39: Report & Summary Delivery
Micro-phase 39.2: Weekly summary
Category: report delivery
Tasks: Aggregate trips; Delays; GPS health; Attendance; Incidents

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 39.3 — Delivery tracking

Parent phase: 39 — Report & Summary Delivery | Category: report delivery

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 39: Report & Summary Delivery
Micro-phase 39.3: Delivery tracking
Category: report delivery
Tasks: Generated; Sent; Failed; Retry

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 40.1 — Alert types

Parent phase: 40 — School Alert Center | Category: alerts

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 40: School Alert Center
Micro-phase 40.1: Alert types
Category: alerts
Tasks: Emergency; Route deviation; Overspeed; GPS offline; Long stop; Delay

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 40.2 — Alert lifecycle

Parent phase: 40 — School Alert Center | Category: alerts

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 40: School Alert Center
Micro-phase 40.2: Alert lifecycle
Category: alerts
Tasks: New; Acknowledged; Resolved

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 40.3 — Incident linkage

Parent phase: 40 — School Alert Center | Category: alerts

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 40: School Alert Center
Micro-phase 40.3: Incident linkage
Category: alerts
Tasks: Create incident from alert; Attach evidence/data; Store resolution

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 41.1 — Help

Parent phase: 41 — Support System | Category: support

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 41: Support System
Micro-phase 41.1: Help
Category: support
Tasks: FAQ; How-to guides; Contact support

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 41.2 — Issue reporting

Parent phase: 41 — Support System | Category: support

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 41: Support System
Micro-phase 41.2: Issue reporting
Category: support
Tasks: Technical issue; Category; Description; Related bus/trip

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 41.3 — Tickets

Parent phase: 41 — Support System | Category: support

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 41: Support System
Micro-phase 41.3: Tickets
Category: support
Tasks: Create; Track; Resolve; History

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 42.1 — Customer operations

Parent phase: 42 — Internal Operations Panel | Category: internal operations

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 42: Internal Operations Panel
Micro-phase 42.1: Customer operations
Category: internal operations
Tasks: School list; School details; Bus count; Customer status

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 42.2 — System health

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 42: Internal Operations Panel
Micro-phase 42.2: System health
Category: internal operations
Tasks: GPS health; WhatsApp delivery health; API health; Error logs

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 42.3 — Support

Parent phase: 42 — Internal Operations Panel | Category: internal operations

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 42: Internal Operations Panel
Micro-phase 42.3: Support
Category: internal operations
Tasks: Support tickets; Activity logs

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 42.4 — Access control

Parent phase: 42 — Internal Operations Panel | Category: internal operations

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 42: Internal Operations Panel
Micro-phase 42.4: Access control
Category: internal operations
Tasks: Internal admin roles; Audit sensitive actions

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 43.1 — Application monitoring

Parent phase: 43 — Monitoring & Reliability | Category: monitoring

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 43: Monitoring & Reliability
Micro-phase 43.1: Application monitoring
Category: monitoring
Tasks: Logs; Error tracking; Health checks

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 43.2 — Infrastructure

Parent phase: 43 — Monitoring & Reliability | Category: monitoring

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 43: Monitoring & Reliability
Micro-phase 43.2: Infrastructure
Category: monitoring
Tasks: Database health; CPU/memory where available; Storage; Backups

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 43.3 — Product monitoring

Parent phase: 43 — Monitoring & Reliability | Category: monitoring

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 43: Monitoring & Reliability
Micro-phase 43.3: Product monitoring
Category: monitoring
Tasks: GPS uptime; WhatsApp failures; Notification failures; ETA failures

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 43.4 — Alerts

Parent phase: 43 — Monitoring & Reliability | Category: monitoring

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 43: Monitoring & Reliability
Micro-phase 43.4: Alerts
Category: monitoring
Tasks: Critical error alerts; Database alerts; Service downtime alerts

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 44.1 — Parent usage

Parent phase: 44 — Analytics | Category: analytics

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 44: Analytics
Micro-phase 44.1: Parent usage
Category: analytics
Tasks: Track requests; ETA requests; Weekly usage

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 44.2 — Driver usage

Parent phase: 44 — Analytics | Category: analytics

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 44: Analytics
Micro-phase 44.2: Driver usage
Category: analytics
Tasks: Start Trip success; End Trip success; Student search; Attendance completion

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 44.3 — System metrics

Parent phase: 44 — Analytics | Category: analytics

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 44: Analytics
Micro-phase 44.3: System metrics
Category: analytics
Tasks: GPS uptime; ETA accuracy; Notification delivery; Delay detection accuracy

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 44.4 — Privacy-aware analytics

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 44: Analytics
Micro-phase 44.4: Privacy-aware analytics
Category: analytics
Tasks: Avoid unnecessary personal data; Define retention; Use aggregated metrics where possible

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 45.1 — Backend unit tests

Parent phase: 45 — Testing — Unit & Integration | Category: testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 45: Testing — Unit & Integration
Micro-phase 45.1: Backend unit tests
Category: testing
Tasks: Trip state; ETA calculations; Delay calculations; Geofence logic; Permissions

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 45.2 — API integration tests

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 45: Testing — Unit & Integration
Micro-phase 45.2: API integration tests
Category: testing
Tasks: Authentication; Bus APIs; Student APIs; GPS APIs; Trip APIs; Notification APIs

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 45.3 — Database tests

Parent phase: 45 — Testing — Unit & Integration | Category: testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 45: Testing — Unit & Integration
Micro-phase 45.3: Database tests
Category: testing
Tasks: Constraints; Spatial queries; Migrations; Seed data

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 46.1 — Driver flow

Parent phase: 46 — Testing — End-to-End | Category: end-to-end testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 46: Testing — End-to-End
Micro-phase 46.1: Driver flow
Category: end-to-end testing
Tasks: Login; Start Trip; GPS; Search student; Boarding; Problem; SOS; End Trip

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 46.2 — Parent flow

Parent phase: 46 — Testing — End-to-End | Category: end-to-end testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 46: Testing — End-to-End
Micro-phase 46.2: Parent flow
Category: end-to-end testing
Tasks: Onboarding; Track bus; ETA; Pickup; Child status; Delay; Absence; Temporary pickup

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 46.3 — School flow

Parent phase: 46 — Testing — End-to-End | Category: end-to-end testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 46: Testing — End-to-End
Micro-phase 46.3: School flow
Category: end-to-end testing
Tasks: Login; Manage bus/driver/student; Route; Live map; Attendance; Alerts; Reports

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 47.1 — Driver device failures

Parent phase: 47 — Failure & Recovery Testing | Category: failure testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 47: Failure & Recovery Testing
Micro-phase 47.1: Driver device failures
Category: failure testing
Tasks: GPS off; Internet off; Low battery; App killed; Phone restart

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 47.2 — Backend failures

Parent phase: 47 — Failure & Recovery Testing | Category: failure testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 47: Failure & Recovery Testing
Micro-phase 47.2: Backend failures
Category: failure testing
Tasks: Backend restart; Database unavailable; Queue/retry behavior

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 47.3 — Integration failures

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 47: Failure & Recovery Testing
Micro-phase 47.3: Integration failures
Category: failure testing
Tasks: WhatsApp delayed; Notification failure; Routing failure

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 47.4 — Data safety

Parent phase: 47 — Failure & Recovery Testing | Category: failure testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 47: Failure & Recovery Testing
Micro-phase 47.4: Data safety
Category: failure testing
Tasks: Duplicate GPS; Duplicate trip start; Duplicate trip end; Invalid/stale location

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 48.1 — Authorization

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 48: Security Testing
Micro-phase 48.1: Authorization
Category: security testing
Tasks: School A cannot access School B; Driver cannot access other bus; Parent cannot access unauthorized student

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 48.2 — Application security

Parent phase: 48 — Security Testing | Category: security testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 48: Security Testing
Micro-phase 48.2: Application security
Category: security testing
Tasks: Input validation; Rate limits; Secure sessions; Secret handling

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 48.3 — Privacy

Parent phase: 48 — Security Testing | Category: security testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 48: Security Testing
Micro-phase 48.3: Privacy
Category: security testing
Tasks: Minimum necessary personal data; Secure tracking links; Retention rules; Audit access

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 48.4 — Backup recovery

Parent phase: 48 — Security Testing | Category: security testing

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 48: Security Testing
Micro-phase 48.4: Backup recovery
Category: security testing
Tasks: Backup creation; Restore test; Document recovery steps

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 49.1 — Pilot configuration

Parent phase: 49 — Pilot Preparation | Category: pilot preparation

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 49: Pilot Preparation
Micro-phase 49.1: Pilot configuration
Category: pilot preparation
Tasks: Select 1 school; Configure 1–3 buses; Register real drivers; Register sample/real parent users with permission

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 49.2 — Operational readiness

Parent phase: 49 — Pilot Preparation | Category: pilot preparation

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 49: Pilot Preparation
Micro-phase 49.2: Operational readiness
Category: pilot preparation
Tasks: Driver training; School admin training; Parent onboarding instructions; Support contact

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 49.3 — Pilot dashboards

Parent phase: 49 — Pilot Preparation | Category: pilot preparation

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 49: Pilot Preparation
Micro-phase 49.3: Pilot dashboards
Category: pilot preparation
Tasks: GPS uptime; ETA accuracy; Trip-start success; WhatsApp delivery; Attendance completion; Parent usage

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 50.1 — Live pilot

Parent phase: 50 — Pilot & Improvement Loop | Category: pilot

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 50: Pilot & Improvement Loop
Micro-phase 50.1: Live pilot
Category: pilot
Tasks: Run real trips; Collect issues; Monitor reliability

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
For UI work, use the existing design system and reusable components. Prefer calm enterprise transport UI: clear hierarchy, compact data density, restrained surfaces, subtle motion, strong map/table/timeline patterns, responsive layouts and accessible contrast. Use ThreeUI only as visual inspiration; do not clone it.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 50.2 — Bug triage

Parent phase: 50 — Pilot & Improvement Loop | Category: pilot

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 50: Pilot & Improvement Loop
Micro-phase 50.2: Bug triage
Category: pilot
Tasks: P0 safety/system outage; P1 major feature; P2 normal bug; P3 minor UI

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 50.3 — Iteration

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 50: Pilot & Improvement Loop
Micro-phase 50.3: Iteration
Category: pilot
Tasks: Fix; Regression test; Deploy; Verify with real users

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 50.4 — Acceptance

Parent phase: 50 — Pilot & Improvement Loop | Category: pilot

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 50: Pilot & Improvement Loop
Micro-phase 50.4: Acceptance
Category: pilot
Tasks: GPS target >95% usable uptime; Initial ETA target roughly ±5–10 minutes; High trip-start reliability; Usable driver workflow; Useful parent alerts

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 51.1 — Performance

Parent phase: 51 — Production Hardening | Category: production hardening

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 51: Production Hardening
Micro-phase 51.1: Performance
Category: production hardening
Tasks: Database indexes; Query optimization; Caching; Pagination

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 51.2 — Reliability

Parent phase: 51 — Production Hardening | Category: production hardening

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 51: Production Hardening
Micro-phase 51.2: Reliability
Category: production hardening
Tasks: Retries; Idempotency; Queues where needed; Graceful degradation

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 51.3 — Operations

Parent phase: 51 — Production Hardening | Category: production hardening

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 51: Production Hardening
Micro-phase 51.3: Operations
Category: production hardening
Tasks: Automated backups; Restore drills; Monitoring; Incident runbook

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 51.4 — Deployment

Parent phase: 51 — Production Hardening | Category: production hardening

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 51: Production Hardening
Micro-phase 51.4: Deployment
Category: production hardening
Tasks: Staging-to-production process; Versioning; Rollback plan; Environment separation

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 52.1 — Hardware abstraction

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 52: GPS Hardware Integration — Later
Micro-phase 52.1: Hardware abstraction
Category: GPS hardware later
Tasks: Define common GPS ingestion contract; Accept phone GPS source; Prepare hardware GPS source

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 52.2 — AIS-140/VLTD evaluation

Parent phase: 52 — GPS Hardware Integration — Later | Category: GPS hardware later

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 52: GPS Hardware Integration — Later
Micro-phase 52.2: AIS-140/VLTD evaluation
Category: GPS hardware later
Tasks: Select compliant hardware options; Evaluate data protocol; Evaluate SIM/network; Evaluate backup power

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 52.3 — Integration

Parent phase: 52 — GPS Hardware Integration — Later | Category: GPS hardware later

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 52: GPS Hardware Integration — Later
Micro-phase 52.3: Integration
Category: GPS hardware later
Tasks: Hardware → same GPS API; Device identification; Health monitoring; Fallback/last-known behavior

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 53.1 — Routing scale

Parent phase: 53 — Scale & Cost Optimization — Later | Category: scale/cost optimization

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 53: Scale & Cost Optimization — Later
Micro-phase 53.1: Routing scale
Category: scale/cost optimization
Tasks: Evaluate self-hosted OSRM/Valhalla; Measure routing cost; Move only when justified

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 53.2 — Infrastructure scale

Parent phase: 53 — Scale & Cost Optimization — Later | Category: scale/cost optimization

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 53: Scale & Cost Optimization — Later
Micro-phase 53.2: Infrastructure scale
Category: scale/cost optimization
Tasks: Dedicated database if needed; Queues; Redis; Load handling; Backups

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 53.3 — Provider abstraction

Parent phase: 53 — Scale & Cost Optimization — Later | Category: scale/cost optimization

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 53: Scale & Cost Optimization — Later
Micro-phase 53.3: Provider abstraction
Category: scale/cost optimization
Tasks: Maps provider switch; Notification provider switch; Storage provider switch

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 53.4 — Cost monitoring

Parent phase: 53 — Scale & Cost Optimization — Later | Category: scale/cost optimization

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 53: Scale & Cost Optimization — Later
Micro-phase 53.4: Cost monitoring
Category: scale/cost optimization
Tasks: Track cloud usage; Track API usage; Track message usage; Identify expensive operations

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 54.1 — Subscription model

Parent phase: 54 — Payments & Commercial Layer — LAST | Category: payments later

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 54: Payments & Commercial Layer — LAST
Micro-phase 54.1: Subscription model
Category: payments later
Tasks: School subscription; Bus count pricing; Monthly billing

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 54.2 — Payments

Parent phase: 54 — Payments & Commercial Layer — LAST | Category: payments later

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 54: Payments & Commercial Layer — LAST
Micro-phase 54.2: Payments
Category: payments later
Tasks: Payment gateway integration; Payment status; Webhook handling

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 54.3 — Invoices

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 54: Payments & Commercial Layer — LAST
Micro-phase 54.3: Invoices
Category: payments later
Tasks: Invoice generation; Payment history; Renewal

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 54.4 — Account controls

Parent phase: 54 — Payments & Commercial Layer — LAST | Category: payments later

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 54: Payments & Commercial Layer — LAST
Micro-phase 54.4: Account controls
Category: payments later
Tasks: Expiry handling; Grace period; Restrictions after non-payment

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.
- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

MICRO-PHASE 54.5 — Commercial analytics

Parent phase: 54 — Payments & Commercial Layer — LAST | Category: payments later

COPY-PASTE PROMPT

You are the implementation agent for our School Bus Tracking System.

PROJECT CONTEXT
- Product: school/college bus tracking and transport safety platform for India.
- Users: school admin, driver, parent.
- Parent: WhatsApp-first plus secure live web page; no mandatory parent app.
- Driver: minimal Android app for GPS/background location; daily simple actions through WhatsApp.
- Target stack: Node.js + TypeScript, PostgreSQL + PostGIS, Flutter Android, Next.js/React, secure parent web.
- Free/open-source first; use APIs/services when they improve speed or reliability.
- Core business logic must remain provider-independent through adapters/interfaces.
- Payments/subscriptions/invoices are intentionally postponed until the final commercial phase.
- Do not reintroduce removed scope: bus capacity, bus active/inactive, parent child-selection, one-parent-multiple-children workflow, or mandatory school approval for normal pickup setup.

CURRENT WORK
Phase 54: Payments & Commercial Layer — LAST
Micro-phase 54.5: Commercial analytics
Category: payments later
Tasks: MRR; School count; Bus count; Churn; Payment failures; School onboarding and admin dashboard; Bus, driver, student and parent management; Dynamic parent pickup location; Route creation and pickup sequencing; Minimal driver Android app with background GPS; Driver WhatsApp: Start Trip, End Trip, Student Search, Report Problem, Emergency, Help; Driver boarding/drop attendance; Live GPS and real-time fleet tracking; Parent WhatsApp: Track My Bus, My ETA, My Child, My Pickup, Notifications, Help; Parent secure live tracking page; ETA engine; Pickup and school geofencing; Bus approaching / pickup arrival notifications; Boarding and drop notifications; Smart delay detection and affected-parent notifications; Long-stop detection and reason capture; GPS offline detection and recovery; Route deviation detection; Overspeed detection; Emergency/SOS workflow; Student journey history; Student not going today / absence; Temporary pickup change; School live dashboard and map; Attendance dashboard; Daily and weekly transport reports; School alert center; Support system; Internal operations panel; Monitoring and analytics; Unit, integration, end-to-end, failure and security testing; Pilot deployment and improvement loop; Production hardening; Later GPS hardware integration; Later scale/cost optimization; Payments and commercial layer — LAST; Bus capacity; Bus active/inactive field; Parent child-selection feature requested for removal; One parent WhatsApp number → multiple children workflow; General mandatory school-admin approval for normal pickup setup; CCTV / AI camera; RFID / face recognition; Fuel / payroll / full ERP; Custom GPS hardware manufacturing; iOS driver app in the initial build; Payments/subscriptions/invoices until the final commercial phase

EXECUTION METHOD
1. Inspect the repository and current architecture before editing.
2. Identify completed dependencies and reuse them. Implement only the smallest missing prerequisite if necessary.
3. Write a short plan/artifact before broad code changes.
4. If data is involved: database/model → API/domain logic → UI/mobile/WhatsApp.
5. Add loading, empty, validation, failure, retry, authorization and offline states where relevant.
6. Keep business rules out of presentational components. Use services/hooks/controllers/domain modules.
7. Keep user-visible copy consistent with school transport and safety workflows.
8. Run tests/type-check/lint/build and verify UI/browser/device behavior when relevant.
9. Inspect the diff for scope creep, hardcoded demo behavior, duplication and security/privacy problems.
10. Finish with a concise artifact: summary, files changed, data/API changes, tests, verification evidence, limitations and next dependency.

UI/UX RULES
If UI is involved, use the existing design system rather than inventing a new visual language. Keep the interface functional, responsive, accessible and data-driven.
- Use design tokens for color, spacing, radius, typography, shadows, focus states and motion.
- Prefer reusable primitives over one-off markup.
- Avoid generic AI-generated SaaS appearance, excessive gradients, excessive glassmorphism, giant hero sections inside operational screens, random icons and unnecessary animation.
- Use motion only to explain state changes, navigation, live updates or feedback.
- Maps are functional, not decorative.
- Never use fake data as final implementation. Fixtures are isolated to tests/dev seeds.
- Do not copy ThreeUI source/assets unless licensing permits it; use the site as inspiration/reference.

ACCEPTANCE CRITERIA
- Implement only this micro-phase and its direct dependencies; do not invent unrelated features.
- Inspect the repository first and reuse existing types, services, components and patterns.

- Do not hardcode production data, IDs, routes, student names, API responses or business rules into UI components.
- Use typed models and a clean service/API layer so real backend data can replace fixtures without rewriting UI.
- Implement loading, empty, success, validation and error states.
- Keep secrets in environment variables; never place API keys or credentials in source code.
- Run relevant tests, lint, type-check and build; fix errors before finishing.
- Report exactly what changed, files changed, tests run, verification evidence, limitations and next dependency.

IMPORTANT
If ambiguity materially affects correctness or safety, stop and ask. Otherwise preserve the existing architecture and do not silently redesign the product. Do not mark this micro-phase complete until it is verifiable.

6. Product UI Quality Gate

☐ No page should feel like a static screenshot.

☐ No repeated generic dashboard cards just to fill space.

☐ No fake numbers/charts in production.

☐ No random gradients or decorative 3D in operational screens.

☐ Real data states must be represented: loading, empty, live, delayed, offline, error, unauthorized.

☐ Maps, tables, timelines and alerts should match real transport workflows.

☐ Driver flows must minimize distraction and taps.

☐ Parent flows must be understandable without training.

☐ School admin workflows must support fast scanning and drill-down.

☐ Every reusable component should have a clear reason to exist.

☐ Motion must communicate state or feedback.

☐ Accessibility and responsive behavior are part of the definition of done.

7. Final Delivery Standard

A micro-phase is complete only when the requested behavior exists in the real architecture, is connected to the correct state/data path, handles failures, passes relevant checks and has been visually/behaviorally verified. Generated code alone is not completion.

Antigravity references: Google Antigravity IDE overview and agent documentation. ThreeUI reference: recent browse and CSS/component catalog.